

CPUs & Program Execution

Session Outline

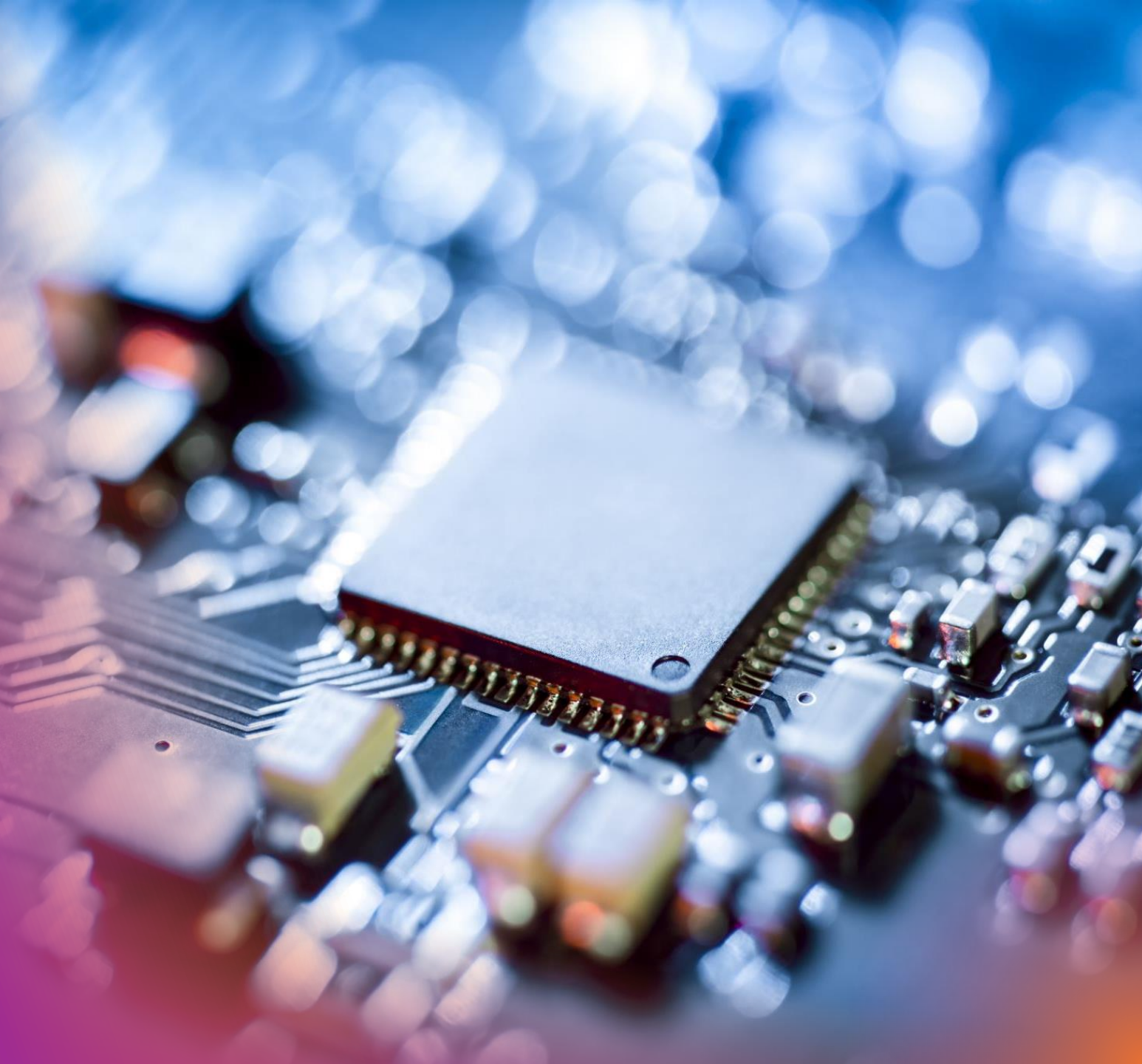
Microprocessor Fundamentals

Register Transfer Language

Assembly Language

Subroutines and Stacks

Addressing Modes



Microprocessor Fundamentals

From Logic Gates to the CPU

- **What we've learned so far:**
- How data is represented digitally (bits, bytes, binary numbers)
- Arithmetic and logic operations using binary
- Logic gates and circuits (AND, OR, NOT, XOR, etc.)
- Combinational and sequential circuits — the building blocks of decision-making
- Counters and flip-flops — introducing *timing* and *state*

How Do These Building Blocks Run a Program?

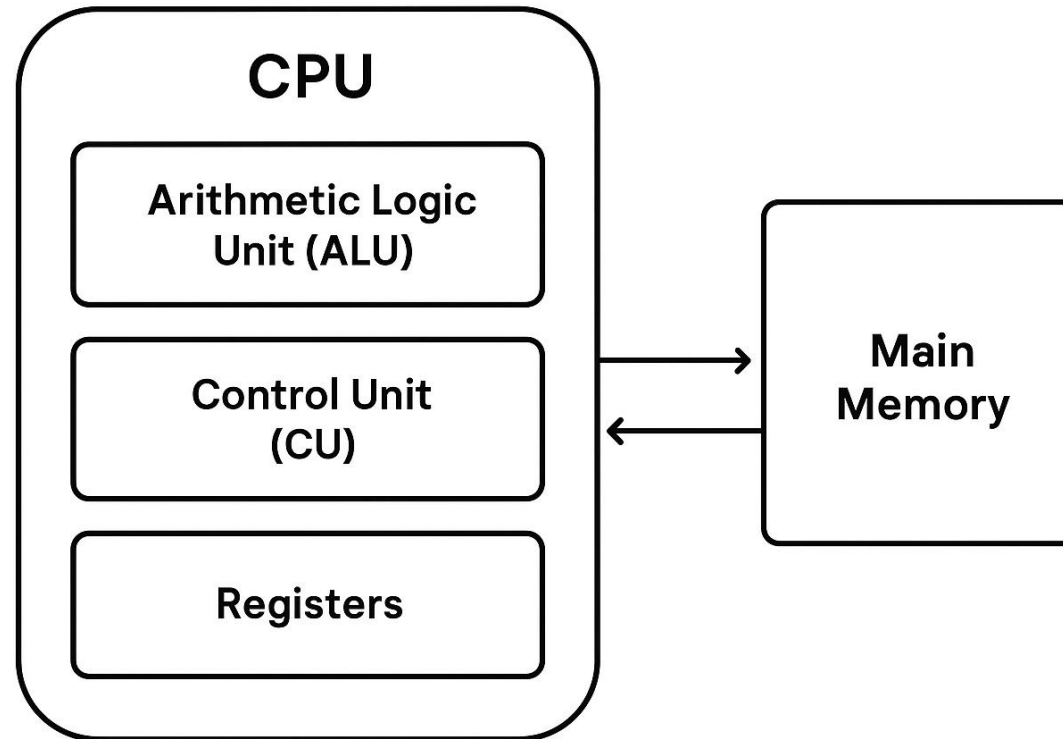
- **This leads us to:**
- The **CPU** — a complex circuit built from billions of logic gates
- Where **arithmetic**, **logic**, and **control** circuits work together
- Turning binary instructions into **meaningful actions**



Central Processing Unit (CPU)

- Controls and performs instructions.
- Modern CPUs are commonly housed on a silicon chip known as a microprocessor.
- Microprocessor-based CPU implementations dominate modern computing.
- Although complex in design, a CPU is typically viewed as having three key components:
 - **Arithmetic Logic Unit (ALU):** Performs mathematical and logical operations.
 - **Control Unit (CU):** Decodes program instructions and manages their execution.
 - **Registers:** Provide high-speed storage for temporary data and instructions.

Central Processing Unit (CPU)





Microprocessor Fundamentals

Fetch Decode Execute Cycle

Fundamentals of CPU Operation

- The CPU continuously performs an instruction cycle.
- Computational instructions are retrieved from memory, decoded into recognizable operations, and executed to affect the CPU's current state.
- This process is known as the fetch–decode–execute cycle.
- Each instruction cycle occurs over multiple CPU clock cycles.
- Recall the importance of clock signals in coordinating sequential logic circuits.
- The fetch–decode–execute cycle relies on the coordinated interaction of several CPU components — including the ALU, CU, and registers.



Microprocessor Fundamentals

The Fetch Cycle

Understanding Machine Code and the Fetch Cycle

- Address (left column): Shows the memory address (in hexadecimal) where each instruction is stored.
- Binary Instructions (middle column): The machine code stored at that memory location — these are the actual bytes fetched by the CPU.
- Assembly Instructions (right column): The human-readable form of the binary instructions.

Binary instructions (Hex code)		Assembly instructions
000000000000011ef <main>:		
11ef:	55	push %rbp
11f0:	48 89 e5	mov %rsp,%rbp
11f3:	48 83 ec 20	sub \$0x20,%rsp
11f7:	89 7d ec	mov %edi,-0x14(%rbp)
11fa:	48 89 75 e0	mov %rsi,-0x20(%rbp)
11fe:	ba 0f 00 00 00	mov \$0xf,%edx
1203:	be 0a 00 00 00	mov \$0xa,%esi
1208:	bf 05 00 00 00	mov \$0x5,%edi
120d:	e8 33 ff ff ff	callq 1145 <function1>
1212:	89 45 fc	mov %eax,-0x4(%rbp)
1215:	48 8b 45 e0	mov -0x20(%rbp),%rax
1219:	48 83 c0 08	add \$0x8,%rax
121d:	48 8b 00	mov (%rax),%rax
1220:	48 89 c2	mov %rax,%rdx
1223:	be 08 00 00 00	mov \$0x8,%esi
1228:	bf 14 00 00 00	mov \$0x14,%edi
122d:	e8 35 ff ff ff	callq 1167 <function2>
1232:	89 45 f8	mov %eax,-0x8(%rbp)
1235:	8b 55 f8	mov -0x8(%rbp),%edx
1238:	8b 45 fc	mov -0x4(%rbp),%eax
123b:	89 d6	mov %edx,%esi
123d:	89 c7	mov %eax,%edi
123f:	e8 6e ff ff ff	callq 11b2 <function3>
1244:	b8 00 00 00 00	mov \$0x0,%eax
1249:	c9	leaveq
124a:	c3	retq
124b:	0f 1f 44 00 00	nopl 0x0(%rax,%rax,1)

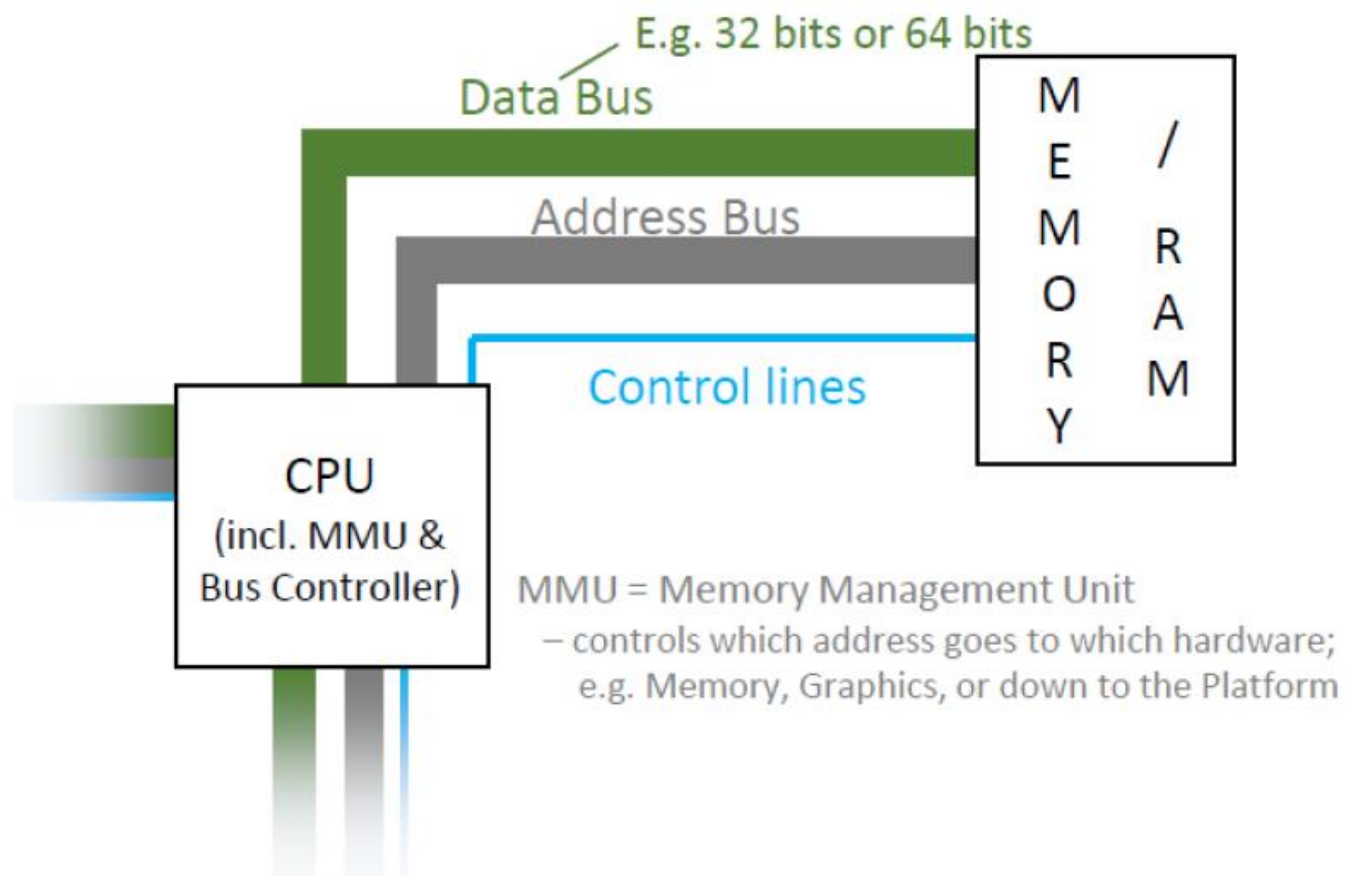
The Fetch Cycle: How the CPU Retrieves Instructions

- The fetch cycle is the first step in the CPU's operation cycle.
- The Program Counter (PC) holds the *address* of the next instruction — e.g., here 0000000000000011ef.
- This address is sent through the Address Bus to memory.
- The Control Unit issues a *Read* signal.
- The Memory places the instruction (55 → push %rbp) on the Data Bus.
- The Instruction Register (IR) stores this fetched instruction for decoding.

Binary instructions (Hex code)		Assembly instructions
00000000000011ef <main>:		
11ef:	55	push %rbp
11f0:	48 89 e5	mov %rsp,%rbp
11f3:	48 83 ec 20	sub \$0x20,%rsp
11f7:	89 7d ec	mov %edi,-0x14(%rbp)
11fa:	48 89 75 e0	mov %rsi,-0x20(%rbp)
11fe:	ba 0f 00 00 00	mov \$0xf,%edx
1203:	be 0a 00 00 00	mov \$0xa,%esi
1208:	bf 05 00 00 00	mov \$0x5,%edi
120d:	e8 33 ff ff ff	callq 1145 <function1>
1212:	89 45 fc	mov %eax,-0x4(%rbp)
1215:	48 8b 45 e0	mov -0x20(%rbp),%rax
1219:	48 83 c0 08	add \$0x8,%rax
121d:	48 8b 00	mov (%rax),%rax
1220:	48 89 c2	mov %rax,%rdx
1223:	be 08 00 00 00	mov \$0x8,%esi
1228:	bf 14 00 00 00	mov \$0x14,%edi
122d:	e8 35 ff ff ff	callq 1167 <function2>
1232:	89 45 f8	mov %eax,-0x8(%rbp)
1235:	8b 55 f8	mov -0x8(%rbp),%edx
1238:	8b 45 fc	mov -0x4(%rbp),%eax
123b:	89 d6	mov %edx,%esi
123d:	89 c7	mov %eax,%edi
123f:	e8 6e ff ff ff	callq 11b2 <function3>
1244:	b8 00 00 00 00	mov \$0x0,%eax
1249:	c9	leaveq
124a:	c3	retq
124b:	0f 1f 44 00 00	nopl 0x0(%rax,%rax,1)

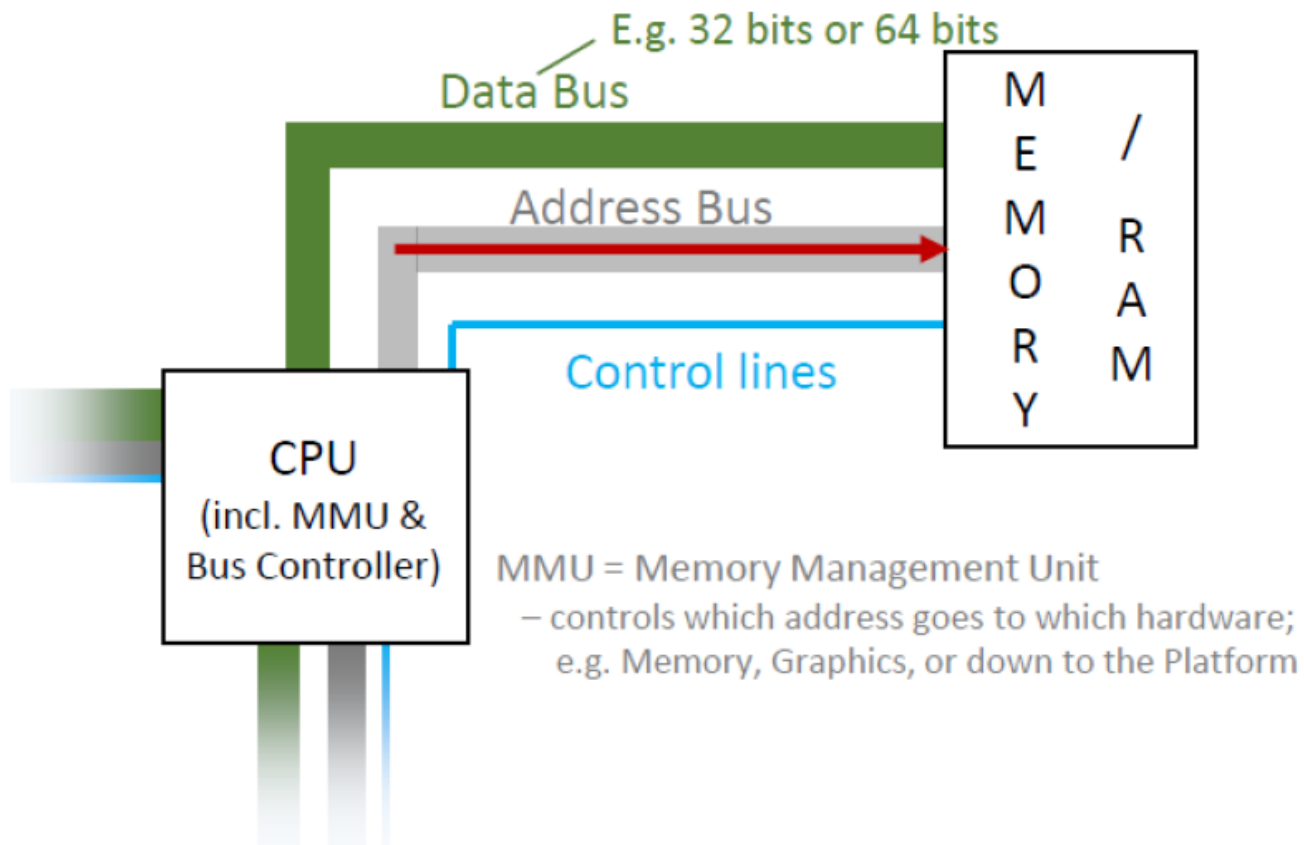
The Fetch Cycle: How the CPU Retrieves Instructions

Computer Architecture – The HW layout:



The Fetch Cycle: How the CPU Retrieves Instructions

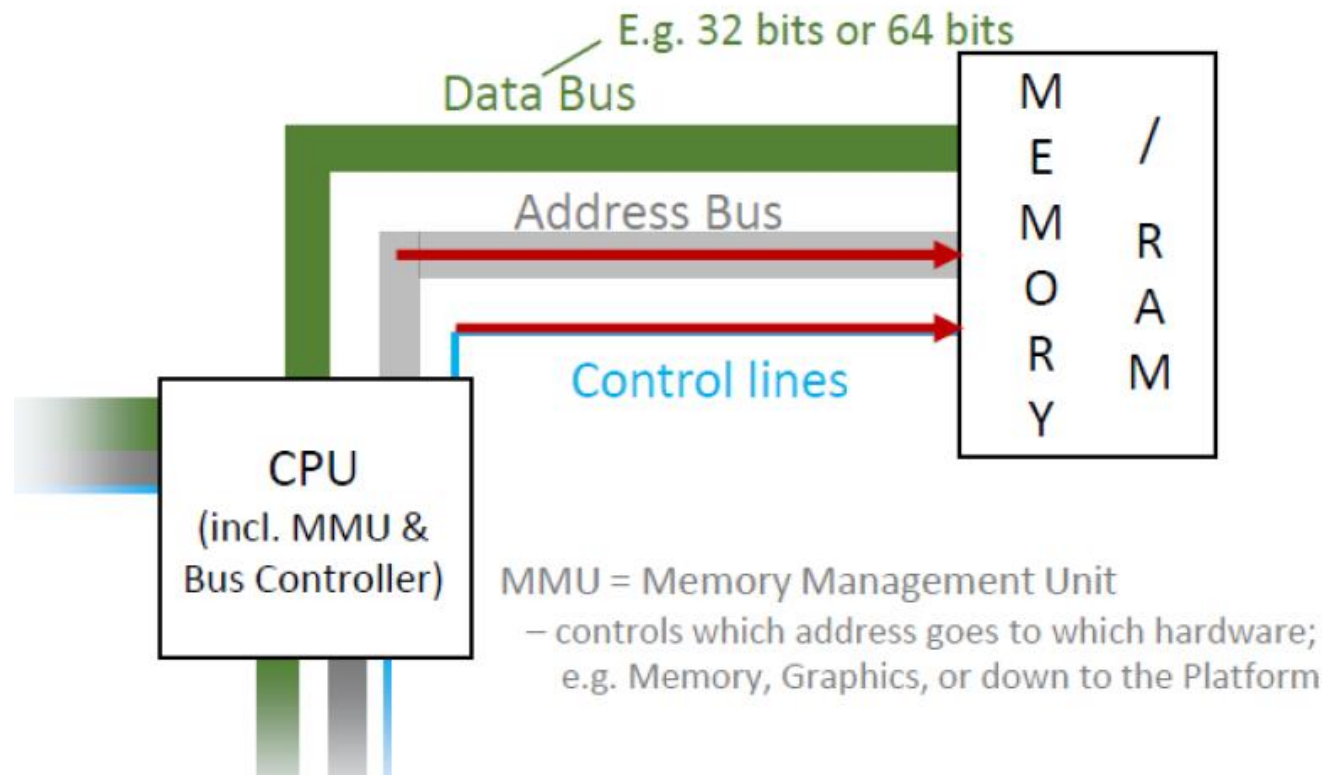
Computer Architecture – The HW layout:



CPU puts the address of the next word of memory onto the address bus

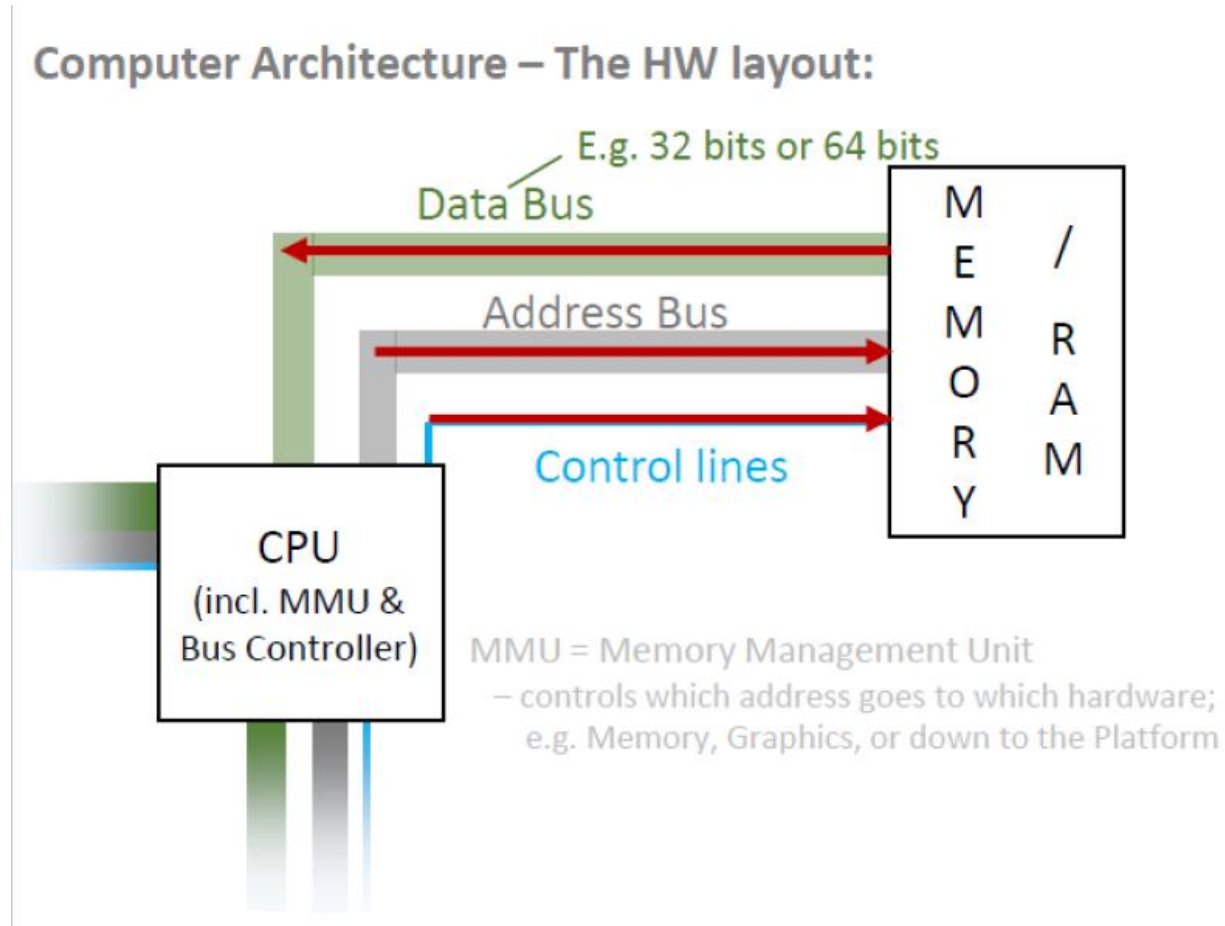
The Fetch Cycle: How the CPU Retrieves Instructions

Computer Architecture – The HW layout:



CPU puts the address of the next word of memory onto the address bus
And the control line is set to read

The Fetch Cycle: How the CPU Retrieves Instructions

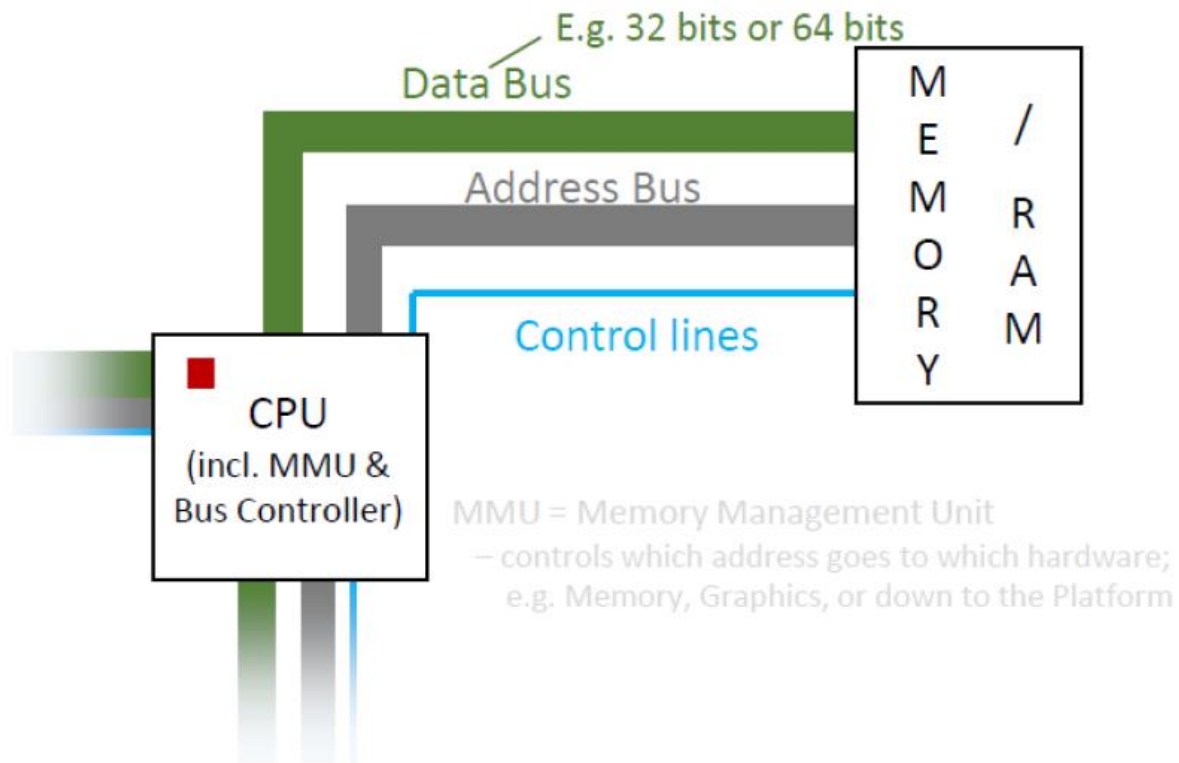


CPU puts the address of the next word of memory onto the address bus
And the control line is set to read
The RAM control circuits push a word of data onto the Data bus, which contains the next machine instruction to be executed

This instruction is called **Fetch** of the instruction

The Fetch Cycle: How the CPU Retrieves Instructions

Computer Architecture – The HW layout:



FETCH: The instruction is inside the CPU



Microprocessor Fundamentals

The Decode Cycle

The Decode Cycle: Interpreting the Instruction

- After the instruction is fetched from memory, the CPU begins to decode it.
- The Control Unit (CU) analyses the binary instruction to understand what action the CPU needs to perform.
- It identifies which parts of the CPU (like the arithmetic circuits or data paths) will be involved.
- The CU then sends the necessary control signals to prepare the system for the next step — execution.

The Decode Cycle: Interpreting the Instruction

- During the decode cycle, the CPU examines the binary instruction fetched from memory.
- The first few bits of this instruction form the Opcode (Operation Code).
- The Opcode tells the CPU what operation to perform — for example, add, move, or jump.
- The remaining bits of the instruction identify where to get the data (operands), such as from memory or internal circuits.

Understanding Opcodes with Examples

Example: `ADD R1, R2`

Meaning: $R1 \leftarrow R1 + R2$

Field	Bits	Description
Opcode	6 bits	000001 (for ADD)
Register 1	3 bits	Destination: R1
Register 2	3 bits	Source: R2

- The **Opcode** tells *what to do*, while the **operand fields** tell *where to do it*.
- More operands mean a longer instruction.
- Control Unit decodes the Opcode to activate ALU for the correct operation.



Microprocessor Fundamentals

The Execute Cycle

The Execute Cycle: Performing the Operation

- Once the instruction is decoded, the CPU carries out the required operation.
- The Control Unit activates the necessary circuits, such as the ALU (Arithmetic Logic Unit), or triggers data movement between memory and CPU.
- For arithmetic or logic instructions, the ALU performs the computation (e.g., addition, comparison).
- For data transfer instructions, data is moved to or from memory or between internal parts of the CPU.

The ALU: The CPU's Calculation Engine

- The Arithmetic Logic Unit (ALU) is the part of the CPU that performs all arithmetic and logical operations.
- It receives control signals from the Control Unit (CU) during the execution phase.
- Common operations include:
 - Arithmetic: Addition, subtraction, multiplication, division
 - Logic: AND, OR, NOT, XOR, comparison (<, >, =)
- The ALU takes its inputs from data lines or registers, performs the operation, and sends the result back for storage or further use.

ALU at Work: Computing $\sin(30^\circ)$ Example

Even trigonometric functions like $\sin()$ are built entirely from basic ALU operations: add, subtract, multiply, divide.

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$30^\circ \times \frac{\pi}{180} = \frac{\pi}{6} \approx 0.5236$$

$$\text{Term 1: } x = 0.5236$$

$$\text{Term 2: } -\frac{x^3}{3!} = -\frac{0.5236^3}{6} \approx -0.0239$$

$$\text{Term 3: } \frac{x^5}{5!} = \frac{0.5236^5}{120} \approx 0.0003$$

$$\sin(30^\circ) \approx 0.5236 - 0.0239 + 0.0003 \approx 0.500$$

Registers – Storing Intermediate Values

- During computation, each term in the series needs to be stored temporarily before final addition.
- Registers provide this fast, temporary storage.

Example (sin(30°) terms):

Register	Stored Value	Purpose
R1	$x = 0.5236$	Stores first term
R2	$-x^3/3! \approx -0.0239$	Stores second term
R3	$x^5/5! \approx 0.0003$	Stores third term
R4	Result ≈ 0.5	Stores final sum

Control Unit Orchestrates Operations

- The Control Unit (CU) directs the CPU to execute a sequence of instructions in order.
- It fetches, decodes, and executes each instruction, ensuring the ALU and registers work together.
- Registers store intermediate results; ALU performs the computation.
- Together, they compute complex operations step by step.



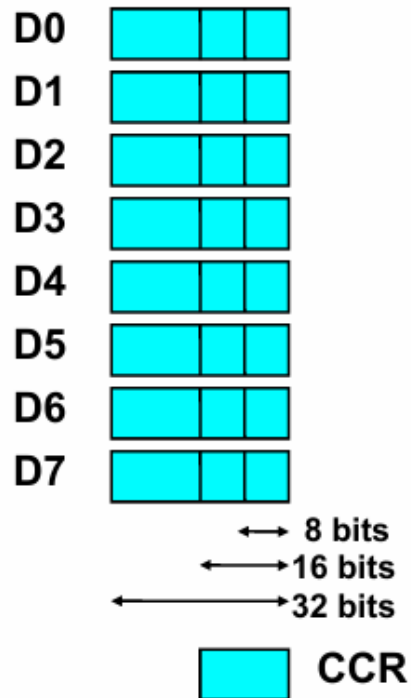
Microprocessor Fundamentals

The 68008 Architecture

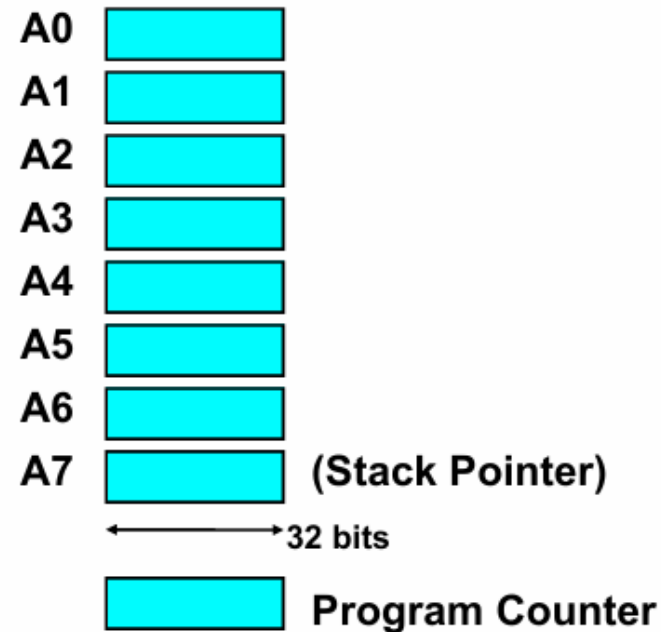
A Worked Example - The 68008 Architecture

To appreciate the general, we will consider a specific example with common characteristics

Data Registers



Address Registers

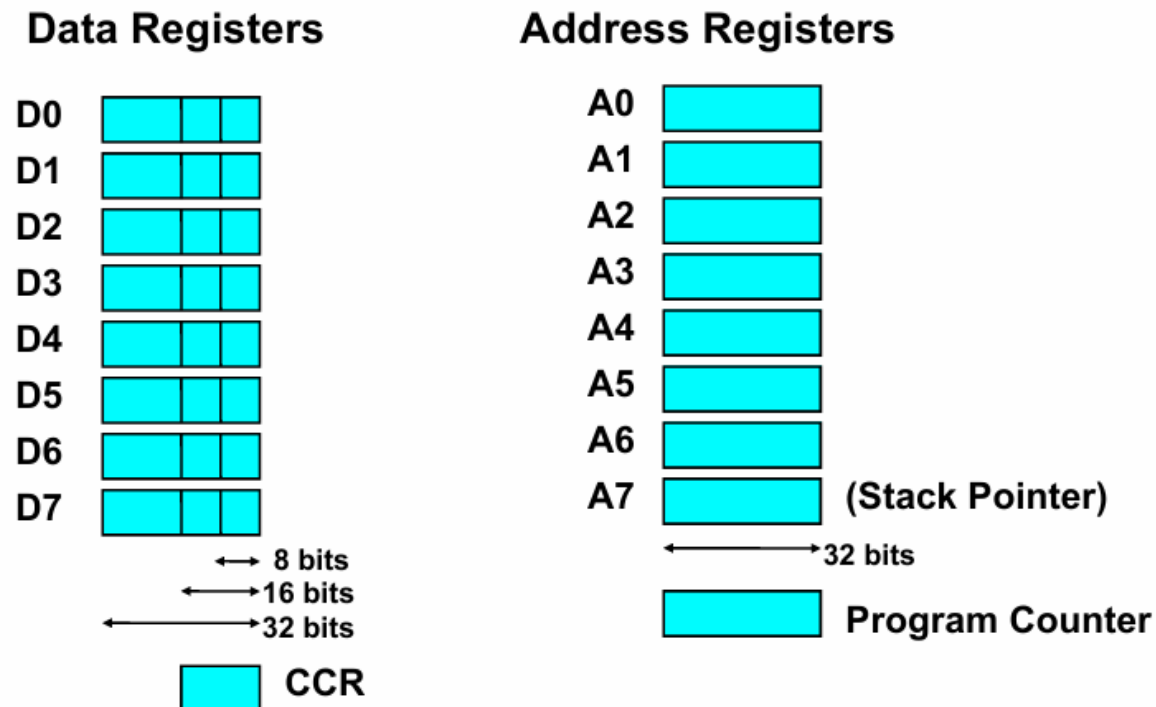


Fetch-Decode-Execute Components

- **Arithmetic Logic Unit (ALU)** - Performs mathematical and logical operations
- **Control Unit (CU)** - Decodes program instructions and handles logistics for the execution of decoded instructions
- **Program Counter (PC)** - Tracks the memory address of the next instruction to be executed
- **Instruction Register (IR)** - Contains most recent instruction fetched
- **Memory Address Register (MAR)** - Contains address of the region of memory to be read or written, i.e., location of data to be accessed
- **Memory Data Register (MDR)** - Contains data fetched from memory or data ready to be written to memory

Programmer's Model of 68008 CPU

Understanding available registers is the cornerstone of a useful programmer's model



Programmer's Model of 68008 CPU

- The programmer's model is a commonly used abstraction, invariably used by assembler programmers, of the internal architecture of a processor
- 68008 processor has identical instruction set to the 68000 processor, but has smaller external buses
 - Internal registers are 32-bits wide
 - Internal data buses are 16-bits wide
 - 68008 has an 8-bit external data bus (16-bit on 68000)
 - 68008 has 20-bit external address bus (24-bit for 68000)

Fetch–Decode–Execute Example: 32-bit Instruction on 68008

Suppose we execute a **32-bit ADD instruction** → ADD.L D1, D2

Cycle	Operation	Details	Impact on Performance
Fetch	Instruction fetched from memory	8-bit external data bus ⇒ CPU fetches 1 byte per cycle → 4 memory cycles needed to fetch 32-bit instruction	Slower instruction fetch (4× slower than 32-bit bus)
Decode	Control Unit decodes opcode and operands	Control Unit interprets the instruction bits and sets ALU control signals	Normal speed (not bus-dependent)
Execute	ALU performs 32-bit addition	ALU is 16-bit , so performs operation in two 16-bit steps (low word → high word)	Takes twice as many ALU cycles
Write Back	Result stored in destination register	16-bit internal data path ⇒ two internal transfers for 32-bit result	Adds minor internal delay

Data Registers

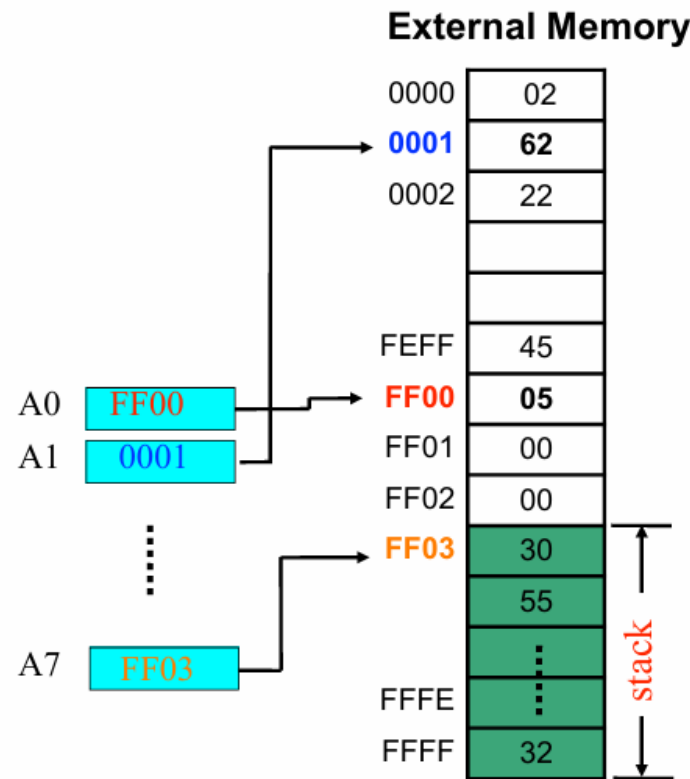
- D0 - D7, 32-bit registers, store frequently used values / intermediate results
- ON CHIP: Strictly speaking we would only need one register on chip - the advantage of many data registers is that fewer references to external memory are required
- Registers can be treated as long, word or byte
 - Long - 32-bits
 - Word - 16-bits (lowest 16 bits)
 - Byte - 8-bits (lowest 8 bits)

Address Registers

- A0 - A6: Used as POINTER REGISTERS in the calculation of operand addresses
- A7 - Additionally used by the processor as a system stack pointer to hold subroutine return addresses etc...
- Operations on addresses do not alter status register Condition Code Register (CCR)
- ALU has capacity to incur changes in status

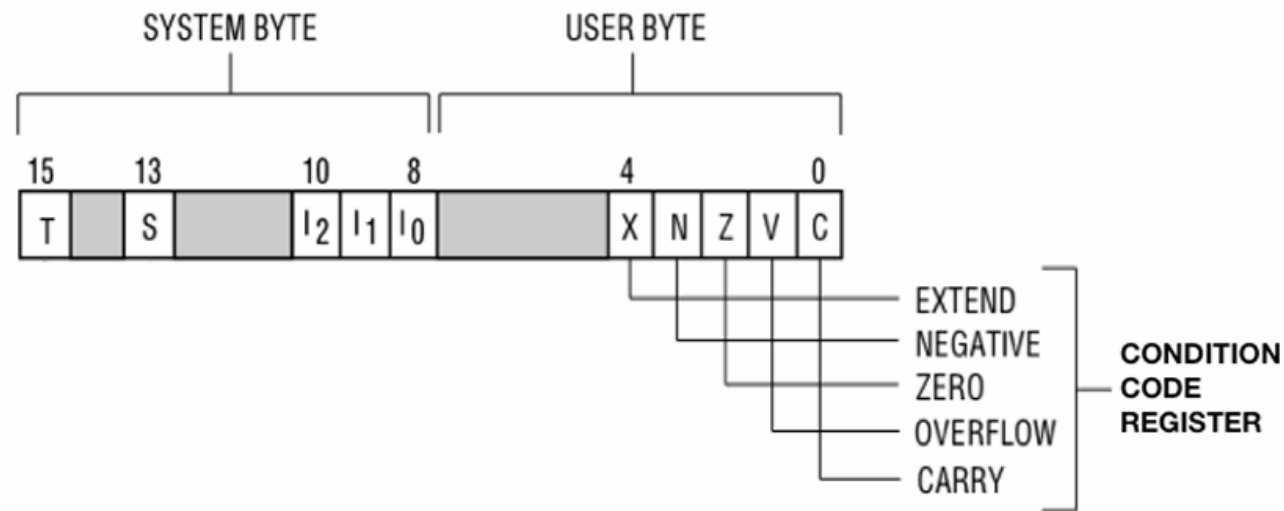
A0-A7, Memory and Stack

- The diagram below illustrates the relationship between address registers, external memory, i.e., not data registers, and the stack



Status Register

- 16 bit - Consists of two 8-bit registers
- Various status bits that are set or reset upon certain conditions arising in the arithmetic and logic unit (ALU)



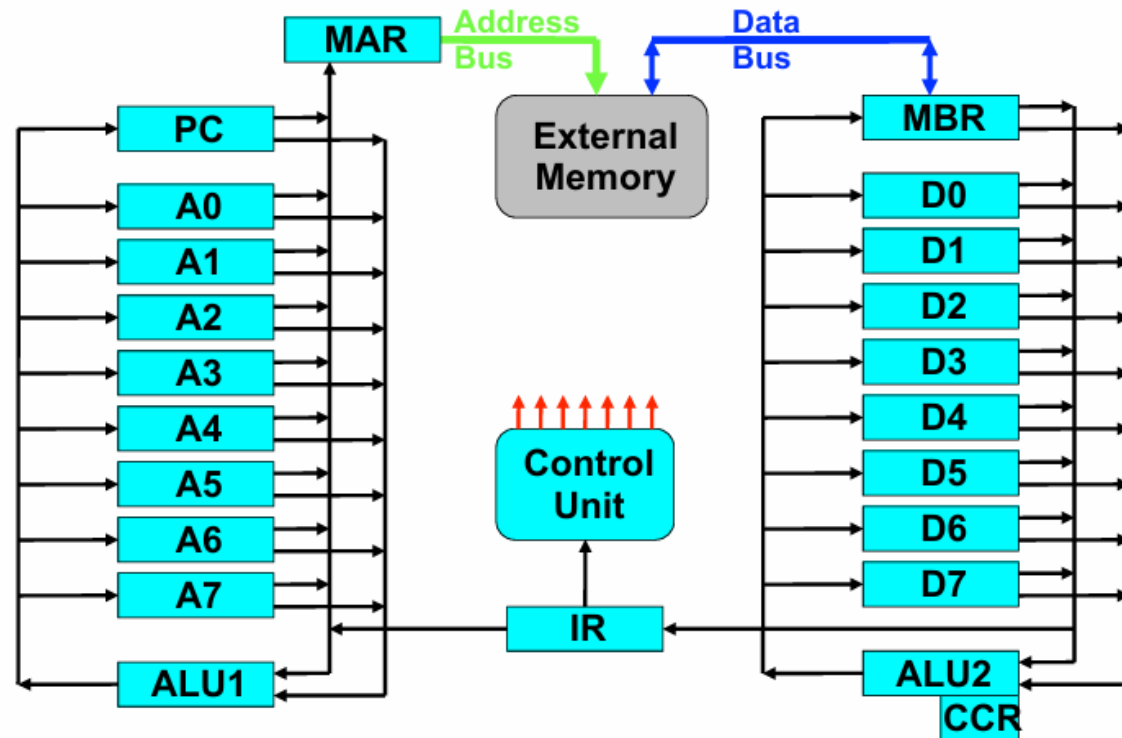
Stack Pointer

- A7 - The stack pointer is used as a pointer into an area of memory called the system stack
- Points to the next free location
- The stack is a Last In First Out (LIFO) structure
- The stack provides temporary storage of essential processor state, e.g., return addresses and registers, during subroutine calls and interrupts

Program Counter (PC)

- A 32-bit register that keeps track of the address at which the next instruction will be found
- In simple terms, it points to the next instruction in memory
- When the current instruction has been read, the PC is incremented to point to the next instruction

68008 Internal Architecture



MAR - Memory Address Register
CCR - Condition Code Register
MBR - Memory Buffer Register
IR - Instruction Register



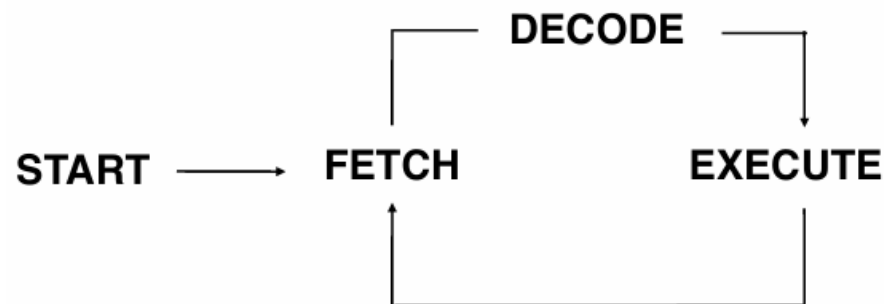
Register Transfer Language

Register Transfer Language

- Used to describe the operations of a microprocessor as it is executing instructions
- For example, $[MAR] \leftarrow [PC]$ means transfer contents of Program Counter to the Memory Address Register
- Computer's memory is called Main Store (MS)
- The contents of memory location 12345 is written $[MS(12345)]$
- Try not to confuse register transfer language with assembler instructions

Instruction Cycle

- We know quite a bit about the instruction cycle and the components involved in the process
- To highlight the power of RTL we can use it to represent stages of the instruction cycle
- The RTL representation we will see makes no attempt to account for the pipelining of instructions
- Pipelining is a common and simple method of speeding up the fetch-execute cycle - you might like to research pipelining in your own time



RTL - Instruction Fetching in Words

1. Contents of Program Counter transferred to MAR address buffers and the Program Counter is incremented
2. MBR loaded from external memory ($R/\overline{W}$ line set to Read)
3. Opcode transferred to Instruction Register from MBR
4. Instruction is decoded

RTL - Instruction Fetching

We can represent the fetch stage of the fetch-execute cycle in RTL

1. $[MAR] \leftarrow [PC]$
2. $[PC] \leftarrow [PC] + 1$
3. $[MBR] \leftarrow [MS([MAR])]$ ($\overline{R/W}$ set to Read)
4. $[IR] \leftarrow [MBR]$
5. $CU \leftarrow [IR(\text{opcode})]$

This part of the cycle is the same for each instruction

RTL - Fetch and Execute

We can represent complex actions, such as adding a constant byte to D0

1. $[MAR] \leftarrow [PC]$
2. $[PC] \leftarrow [PC] + 1$
3. $[MBR] \leftarrow [MS([MAR])]$ ($\overline{R}/W$ set to Read)
4. $[IR] \leftarrow [MBR]$
5. $CU \leftarrow [IR(\text{opcode})]$
- ...

RTL - Fetch and Execute

...

6. $[MAR] \leftarrow [PC]$

7. $[PC] \leftarrow [PC] + 1$

8. $[MBR] \leftarrow [MS([MAR])]$

9. $ALU \leftarrow [MBR] + D0$

10. $[D0] \leftarrow ALU$

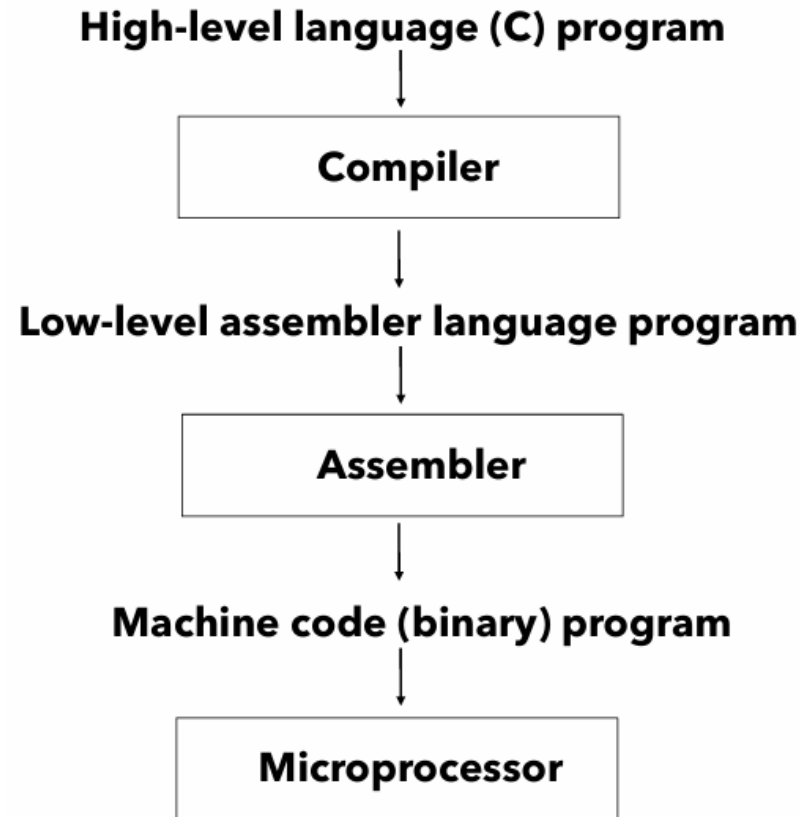


Assembly Language

C Programming

- You can work at the hardware-software interface by using the C programming language
- C is compiled to assembler - the low level instruction set of the microprocessor
- Assembler programs are assembled in order to generate loadable binary
- We concentrate on understanding the instruction set

Compiler, Assembler and Microprocessor



Assembly Language

- Rather than programming in machine code, i.e., placing numbers in memory locations, we prefer to program at a slightly higher level, i.e., in an assembler language
- Assembler language uses easily remembered mnemonics for each instruction, e.g., MOVE D0, D1
- Assembler language also allows memory locations and constants to be given symbolic names
- A point in a program can be referred to by its name rather than a numeric address

Machine and Assembler Codes

Microprocessors "understand" programs of 0's and 1's

For example:

1010 1001 A9

0000 0101 05

Hex is an aid, but what does the following small program do?

D8 A2 FF 9A 18 A9 05 69 07 8D 11 00

This is why we use assembly languages

Assembler Format

Assembly languages vary but typically have a similar format:

<LABEL>: <OPCODE> <OPERAND(S)> | COMMENT

For example:

```
START: move.b    #5, D0    | load D0  
                        | with 5
```

Example Assembler Program

Understanding assembler programs is about recognising each small operation and how it fits as part of a larger sequence of operations

ORG	\$4B0	this program starts at hex 4B0
move.b	#5, D0	load D0 with 5
add.b	#\$A, D0	add 10 to D0
move.b	D0, ANS	store result in ANS
ANS:	DS.B 1	leave 1 byte of memory empty and give it the name ANS

Numbers:

: indicates a constant. A number without # prefix is an address

Default number base is DECIMAL

\$: means in HEX

% : means in BINARY

Assembler Directives:

ANS: A label, i.e., a symbolic name, that must be terminated by a colon

DS (Define Storage): instructs the assembler to reserve some memory

ORG (Origin): tells the assembler where in memory to start putting the instructions or data

68008 Instruction Set

- There are two aspects to the 68008 instruction set:
- **Instructions** - The commands that tell the processor what operations to perform
- **Addressing Modes** - The ways in which the processor can access data or memory locations, i.e., the ways in which addresses may be calculated by the CPU

68008 Instructions

The 68008-instruction set is made up of five categories of instructions:

1. Data Movement
2. Arithmetic
3. Logical
4. Branch
5. System Control

We will look briefly at the first four groups

Form of 68008 Assembler Instructions

Assembler instructions are written in the form:

operation.datatype source, destination

The operation can be on one of three data types:

byte .b (8 bits)

word .w (2 bytes)

long word .l (4 bytes)

The data type and dot may be omitted if the data type is word

Data Movement Instructions

move.b	D0, D1	[D1(0:7)] ← [D0(0:7)]
moveb	D0, D1	the same
move.w	D0, D1	[D1(0:15)] ← [D0(0:15)]
move	D0, D1	the same
move.l	\$F20, D3	[D3(24:31)] ← [MS(\$F20)]
		[D3(16:23)] ← [MS(\$F21)]
		[D3(8:15)] ← [MS(\$F22)]
		[D3(0:7)] ← [MS(\$F23)]
		(Big-Endian mean this way around)
exg.b	D4, D5	exchange
swap	D2	swap lower and upper words
lea	\$F20, A3	load effective address [A3] ← [\$F20]

Arithmetic Instructions

The 68008 does not have hardware floating point support – instructions operate on integers

add.l Di, Dj

| [Dj] $\leftarrow$ [Di] + [Dj]

addx.w Di, Dj

| also add in x bit from CCR

sub.b Di, Dj

| [Dj] $\leftarrow$ [Dj] - [Di]

subx.b Di, Dj

| also subtract x bit from CCR

mulu.w Di, Dj

| unsigned multiplication:

| [Dj(0:31)] $\leftarrow$ [Di(0:15)] * [Dj(0:15)]

muls.w Di, Dj

| signed multiplication

divu.b Di, Dj

divs.l Di, Dj

Logical Instructions

These instructions perform bit-wise operations on data and include AND, OR, EOR, NOT

For example, if register D3 contains 1010 0101, then...

AND.B #11110000, D3

... will produce the result 1010 0000 in the least significant byte of register D3

Examples of Logical Instructions - Logic

Once you understand a few of these operations the operation of the rest becomes much clearer

Logical AND:
AND.B #\$7F, D0

D0	1	1	0	1	1	0	1	0
7F	0	1	1	1	1	1	1	1
D0	0	1	0	1	1	0	1	0

(keep just the lower 7 bits and ignore the MSB)

Logical OR:
OR.B D1, D0

D1	1	0	0	0	0	0	1	0
D0	0	0	0	1	0	1	1	0
D0	1	0	0	1	0	1	1	0

Examples of Logical Instructions - Shifts

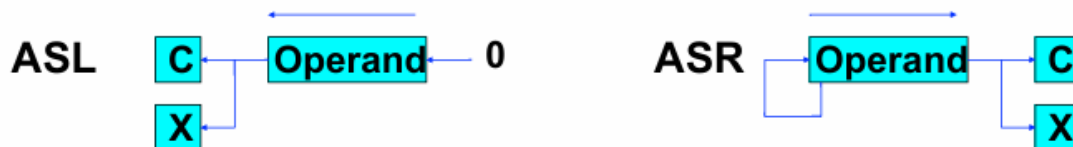
Shift operations are often fundamental to many computational tasks- remember we achieved multiplication by 2 using a left-shift operation

| Logical Shift (**L** - left, **R** - right)



(C and X are in Condition Code Register)

| Arithmetic Shift



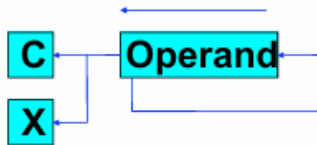
(ASR preserves sign bit)

Examples of Logical Instructions - Rotates

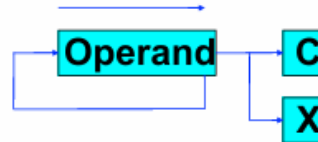
Note the difference between rotating (RO) and rotating through extend bit (ROX) - what does ROX achieve?

| **RO**tate

ROL

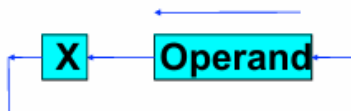


ROR

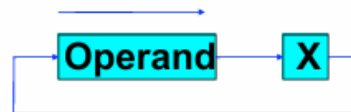


| **RO**tate through e**X**tend bit

ROXL



ROXR



Branch Instructions

Branch instructions cause the processor to branch (jump / GOTO) the labelled address

The instruction can test the state of the CCR bits and branch if a certain condition is met

CCR flags are set by the previous instruction

Form: `Bcc <label>` (where cc is a condition code)

If a branch is taken, $[PC] \leftarrow \text{label}$

Examples of Branch Instructions

15 branch instructions, including:

BRA branch unconditionally

BCC branch on carry clear $(\overline{C})$

BCS branch on carry set (C)

BEQ branch on equal (Z)

BGE branch on greater than or equal $(N.V + \overline{N}.\overline{V})$

...

BPL branch on plus (ie positive)

BVC branch on overflow clear $(\overline{N})$

BVS branch on overflow set $(V)(\overline{V})$



Subroutines and Stacks

Subroutines

Subroutines are useful for frequently used sections of a program

Write (and debug) a subroutine once, and use that program code whenever it is needed

- Reduces program size

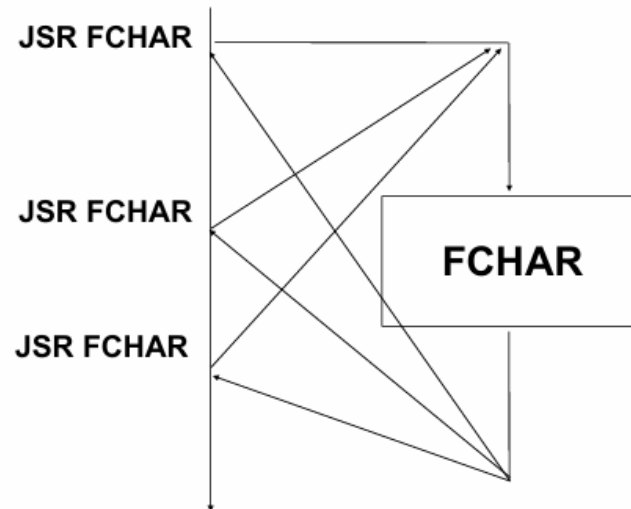
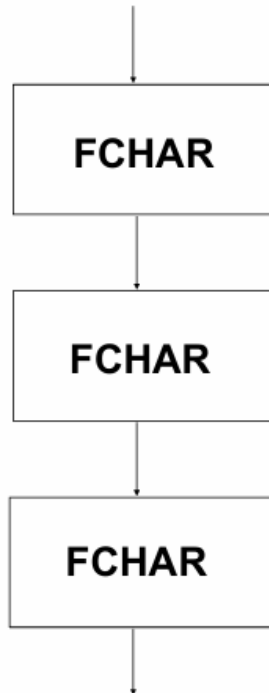
- Improves readability

JSR <label> **Jump to SubRoutine**

RTS **Return from SubRoutine**

Subroutine Example

Can you see how subroutines work here - what assumption do you make when you return from a subroutine?

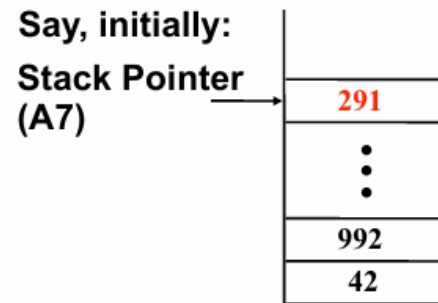


Problem - we need to know where to return

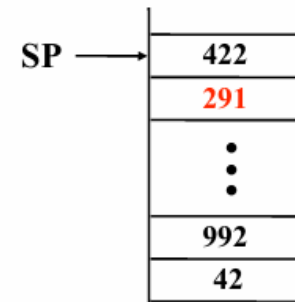
Subroutines and Stacks

A stack can be used to capture the last in first out (LIFO) aspect of the assumption we were making

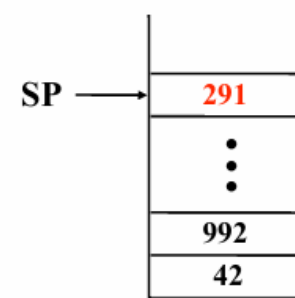
The stack is a last in first out structure. As before, A7 points to the Top Of Stack.



Then push (add) new value:



Then pop (remove) value:



Use of Stack for Subroutine Calls

Subroutine Call (JSR)

- Saves (pushes) the contents of the PC on the stack

- Puts start address of subroutine in PC

Return from Subroutine (RTS)

- Restores (pops) the return address from the stack and puts it in PC

So the stack stores where to return from a subroutine

A subroutine can call another subroutine - we have multiple return addresses on the stack in this case



Addressing Modes

Addressing Modes

How we tell the computer where to find data it needs

Need to organise application data

Some data never changes, some is variable and some needs to be located within a data structure, e.g., list, table or array

In 68008 systems data can be located in a data register, within the instruction itself or in external memory

Addressing modes have expressive power

- Provide data directly

- Specify exactly where data is

- Specify how to go about finding data

68008 Addressing Modes

1. Data or Address Register Direct
2. Immediate Addressing
3. Absolute Addressing
4. Address Register Indirect - five variations
5. Relative Addressing

Data or Address Register Direct

Perhaps the simplest addressing mode to comprehend if you understand the 68008 architecture

The address of an operand is specified by either a data register or an address register

Example 1:

move D3, D2



Example 2:

move D3, A2



Immediate Addressing

The operand forms part of the instruction and remains constant throughout the execution of a program

```
move.b  #$42, D5
```

\$42 → **D5**

The above puts the hex value 42 into register D5

Note the # symbol!

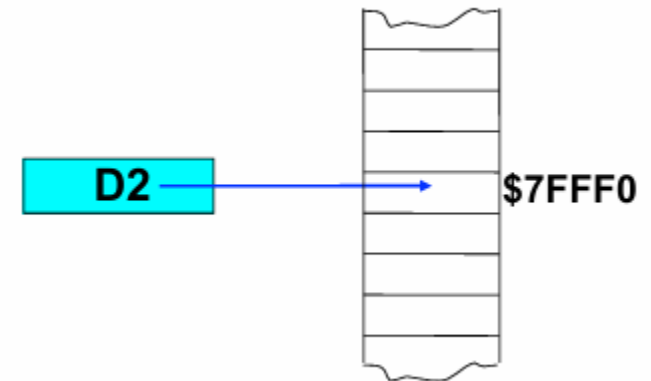
```
[D5] ← $42
```

Absolute Addressing

The operand specifies the location in memory explicitly, meaning that no further processing required

move.l D2, \$7FFF0

[MS(7FFF0)] ← [D2]



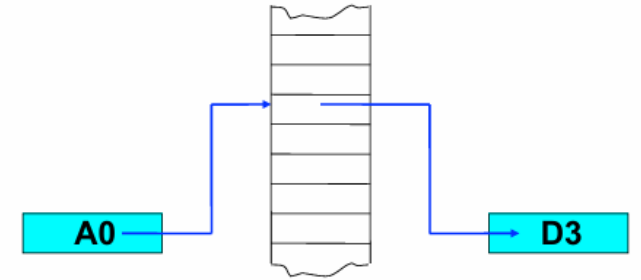
Absolute addressing does not allow position independent code because a program will consistently use the same memory address

Note there is no # symbo

Address Register Indirect

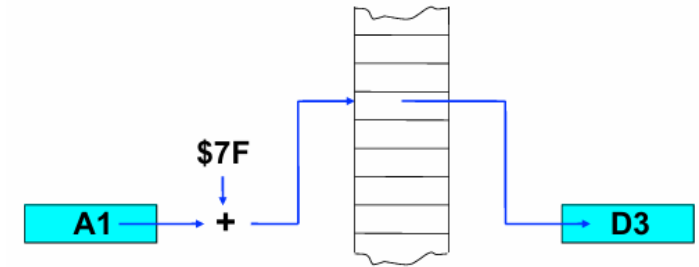
Address Register Indirect

move (A0), D3



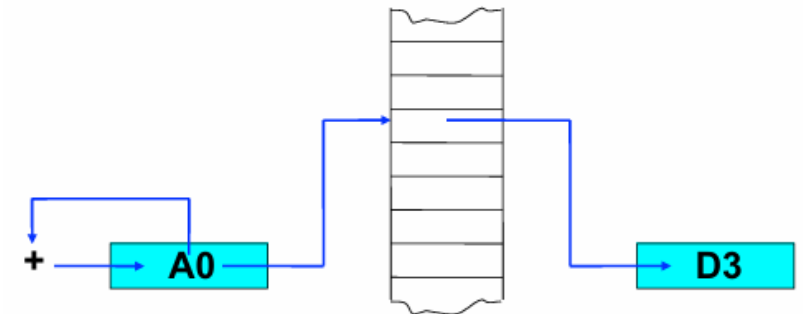
Address Register Indirect with offset

move 7F(A1), D3



Post-Incrementing Address Register Indirect

move.b (A0)+, D3

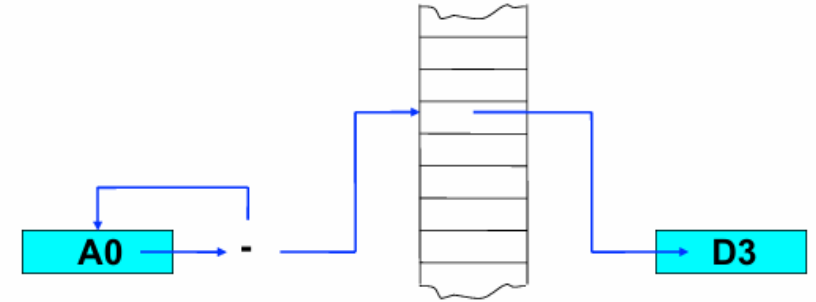


Means take the contents of address register A0, use this number as the address at which the data will be found. Move this data to register D3 and increment A0

Address Register Indirect

Pre-Decrementing Address Register Indirect

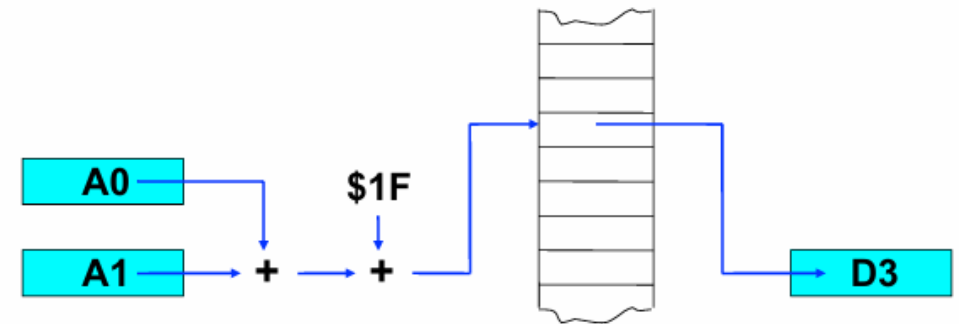
move.b -(A0), D3



Means decrement the contents of address register A0, and use this number as the address at which the data will be found. Move this data to register D3.

Indexed Addressing

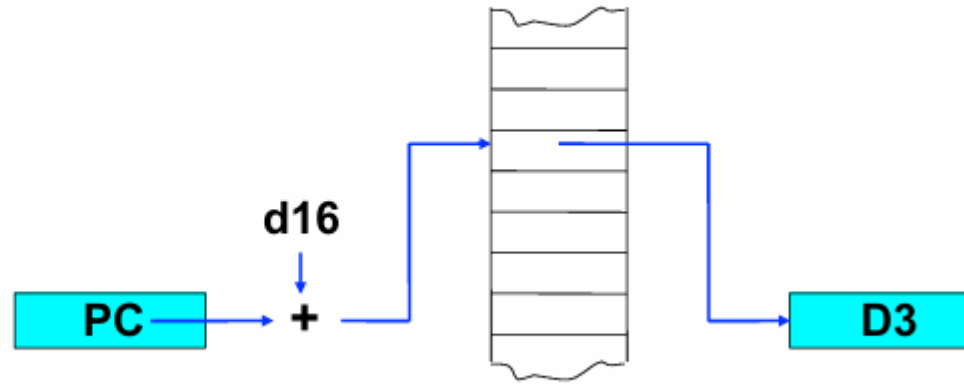
move.l 1F(A0, A1), D3



Means add the contents of address register A0 to A1, add to this the (8-bit two's complement) hex constant 1F and use this number as the address at which the data will be found. Move this data to register D3.

Address Register Indirect

move d16(PC), D3



Code like this contains no absolute addresses, i.e., it uses only addresses relative to the current program counter, and therefore can be placed anywhere in memory

This addressing mode can be used to write “position independent code”